Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen 2do Parcial

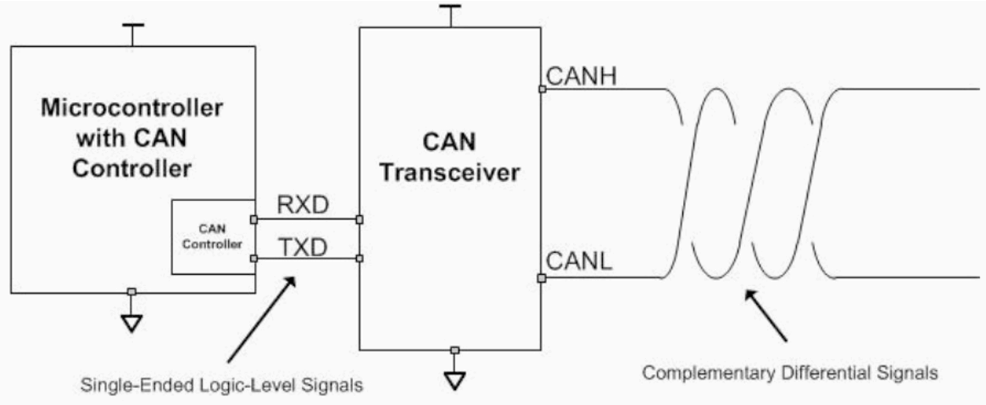
Ignacio Sammartino

Índice

1 Clase 7 2
1. 1 Protocolo CAN (Controller Area Network) 2
2 Clase 8 7
2. 1 Periféricos de Temporización y Módulo FlexTimer (FTM) 7
3 Clase 9 15
3. 1 ADC: Conversor Analógico–Digital 15
3. 2 PIT: Periodic Interrupt Timer 19
3. 3 DMA: Direct Memory Access 21

Clase 7

Protocolo CAN (Controller Area Network)



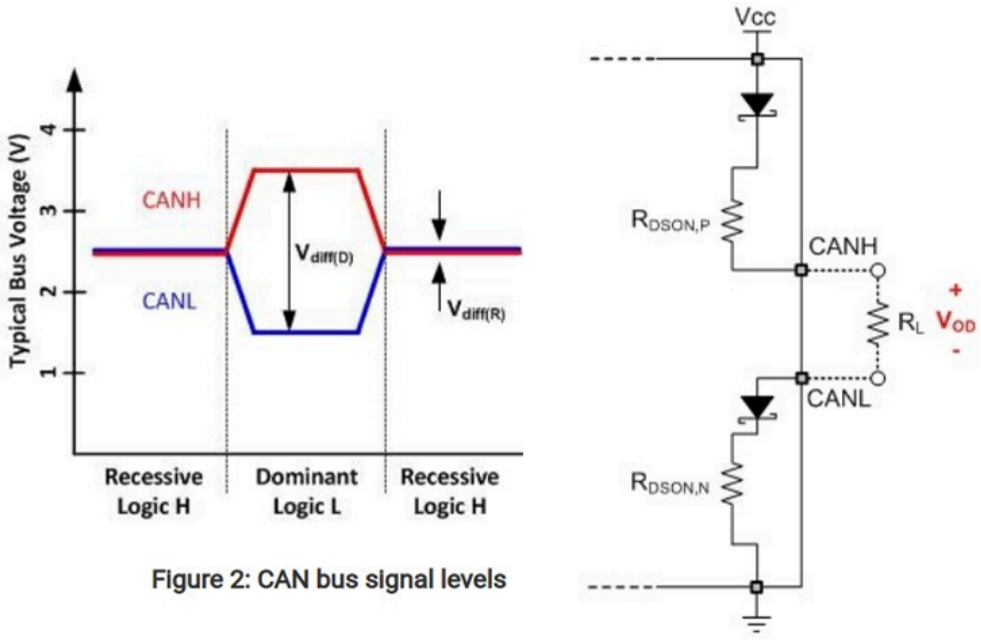


Figure 2: CAN bus signal levels

Arquitectura y Características Principales

CAN es un estándar de bus serie asíncrono, multimaestro y altamente robusto diseñado originalmente por Bosch para aplicaciones automotrices e industriales (ISO 11898). Permite que múltiples Unidades de Control Electrónico (ECUs) se comuniquen sobre un bus común sin necesidad de un nodo maestro central o controlador host.

Características fundamentales del protocolo:

- **Multimaestro / Asíncrono:** Cualquier nodo puede iniciar la transmisión cuando el bus está libre (**Idle**).
- **Orientado a Mensajes:** Los mensajes no contienen direcciones de nodo origen o destino; en su lugar, incorporan un **Identificador (ID)** que define la naturaleza del dato y su prioridad.
- **Inmunidad al Ruido:** Utiliza señalización diferencial sobre par trenzado.
- **Arbitraje No Destructivo:** Resuelve colisiones en tiempo real mediante lógica de bus dominante/recesivo (**Wired-AND**).
- **Detección y Manejo de Errores:** Lógica integrada por hardware para contención y aislamiento de fallas.

Capa Física y Señal Diferencial

A nivel eléctrico, el bus CAN se compone de un par de cables trenzados (**twisted pair**) denominados **CANH** ($\mathrm{CAN}+$ y **CANL** ($\mathrm{CAN}-$, acompañados por una referencia común de masa (GND). Para evitar reflexiones de

onda electromagnética en altas frecuencias, el bus exige **resistencias de terminación de** 120Ω instaladas en ambos extremos físicos de la línea.

El bus opera bajo dos estados eléctricos distintos que implementan la lógica **Wired-AND**:

⚙️ Estado Recesivo (‘1’ Lógico)

- Transceptores en alta impedancia (apagados).
- Las líneas descansan pasivamente a través de resistencias **pull-middle**.
- Voltaje: $V_{\text{CANH}} = 2.5\ \text{V}$, $V_{\text{CANL}} = 2.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = V_{\text{CANH}} - V_{\text{CANL}} = 0\ \text{V}$.

⚙️ Estado Dominante (‘0’ Lógico)

- Transceptores activos excitando la línea.
- Se fuerzan niveles de tensión separados.
- Voltaje: $V_{\text{CANH}} = 3.5\ \text{V}$, $V_{\text{CANL}} = 1.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = 2.0\ \text{V}$.

💡 Lógica Wired-AND y Colisión

Al ser un bus cableado en configuración **Wired-AND**, si dos nodos intentan transmitir simultáneamente y uno de ellos impone un estado **Dominante (‘0’)** mientras el otro envía un **Recesivo (‘1’)**, el estado dominante sobrescribe físicamente al recesivo en todo el bus. Esto permite resolver colisiones sin destruir el mensaje dominante.

Estructura de Trama, Arbitraje y Bit Stuffing

1. 1. 3. 1 Trama de Datos Estándar (Standard Data Frame)
Un **Data Frame** Estándar (CAN 2.0A) se compone de los siguientes campos secuenciales:

Campo	Longitud	Descripción
SOF (Start of Frame)	1 bit	Bit dominante ($1 \to 0$) que marca el inicio de trama y sincroniza a todos los nodos.
Arbitration Field	12 bits	Contiene el **Identificador (ID)** de 11 bits y el bit **RTR** (0 = Data Frame, 1 = Remote Frame).
Control Field	6 bits	Incluye bits de extensión/reserva (`IDE`, `r0`) y el `DLC` (Data Length Code, 4 bits: $0..8$ bytes).
Data Field	$0..8$ bytes	Carga útil (**payload**) del mensaje ($0..64$ bits).
CRC Field	16 bits	Secuencia de verificación de redundancia cíclica de 15 bits + 1 bit `CRC Delimiter` (recesivo).
ACK Field	2 bits	`ACK Slot` (1 bit, enviado recesivo por el emisor y forzado a dominante por receptores con CRC válido) + `ACK Delimiter` (1 bit recesivo).
EOF (End of Frame)	7 bits	Fin de trama compuesto por 7 bits recesivos consecutivos.
ITM (Intermission)	3 bits	Espacio de descanso recesivo obligatorio entre tramas (**Interframe Space**).

1. 1. 3. 2 Arbitraje No Destructivo (CSMA/CD + AMP)
Cuando el bus entra en estado pasivo (**Idle**), cualquier nodo con un mensaje pendiente puede emitir el bit `SOF`. Si dos o más nodos inician la transmisión en el mismo instante:

1. Cada nodo transmite los bits de su `ID` mientras monitorea continuamente el nivel físico del bus.
2. Si un nodo envía un bit **recesivo (‘1’)** en su ID pero lee un nivel **dominante (‘0’)** en el bus, detecta que otro mensaje en competencia tiene mayor prioridad.
3. El nodo perdedor retira su transmisor inmediatamente, pasa a modo de solo escucha (**listening only**) y aguarda a que el bus vuelva a quedar libre.

4. **Regla de prioridad:** Cuanto **menor sea el valor numérico del ID**, mayor es su prioridad en la red (el bit dominante '0' predomina en las primeras posiciones).

1. 1. 3. 3 Regla de Bit Stuffing

Debido a que CAN es un protocolo asíncrono sin línea física de reloj, los receptores dependen de las transiciones de voltaje ($1 \rightarrow 0$) para sincronizar sus relojes internos.

> ⚠️ **Mecanismo de Bit Stuffing**
>
> Si el transmisor detecta **5 bits consecutivos de la misma polaridad** en el flujo de datos, el hardware inserta automáticamente **1 bit de relleno (Stuff Bit)** de polaridad opuesta antes de enviar el siguiente bit. Los nodos receptores remueven este bit de relleno de forma transparente antes de procesar la trama.
>
> Esta regla se aplica estrictamente desde el bit `SOF` hasta el final de la secuencia `CRC` (se excluyen el `CRC Delimiter`, el campo `ACK` y el `EOF`).

Base de Tiempo y Sincronización (Nominal Bit Time)

1. 1. 4. 1 División del Bit en Time Quanta t_q

En redes CAN, la duración de cada bit (**Nominal Bit Time**) se subdivide en N unidades discretas de tiempo denominadas **Time Quanta** (t_q. La duración de 1 t_q se deriva del reloj interno del microcontrolador (f_{osc} mediante el divisor **Baud Rate Prescaler** (BRP.

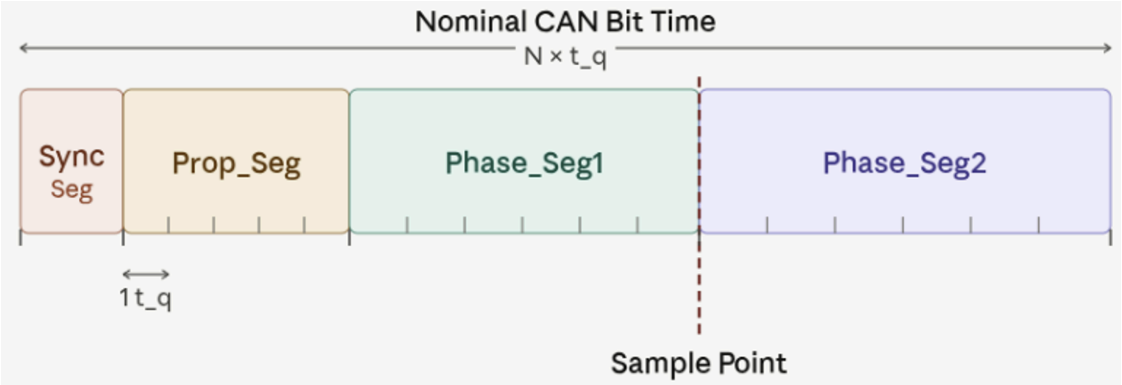


Figura 4: Estructura interna del Nominal Bit Time en sus cuatro segmentos de Time Quanta

El tiempo de bit se divide exactamente en cuatro segmentos configurables:

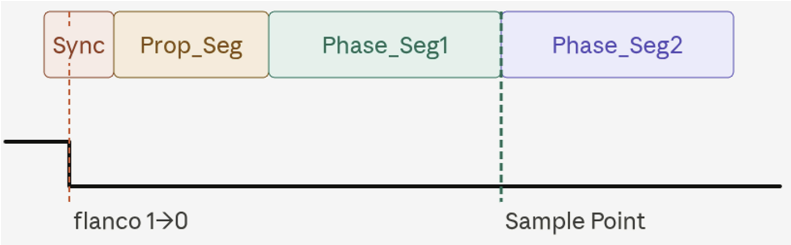
1. **Sync_Seg (Synchronization Segment):** Fijado siempre en 1 t_q. Es el intervalo donde se espera que ocurra el flanco descendente ($1 \rightarrow 0$ que marca el inicio del bit.
2. **Prop_Seg (Propagation Segment):** Configurable de 1 a 8 t_q. Compensa los retrasos físicos de propagación de las ondas electromagnéticas a lo largo del cable y los retardo internos de los transceptores.
3. **Phase_Seg1 (Phase Buffer Segment 1):** Configurable de 1 a 8 t_q. Se ubica inmediatamente antes del **Sample Point**. Se alarga en la resincronización si el flanco llega tarde.
4. **Phase_Seg2 (Phase Buffer Segment 2):** Configurable de 1 a 8 t_q. Se ubica inmediatamente después del **Sample Point**. Se acorta en la resincronización si el flanco llega temprano.

> 💡 **Sample Point (Punto de Muestreo)**
>
> El **Sample Point** es el instante exacto, situado justo al final de `Phase_Seg1`, en el que el nodo evalúa el estado lógico del bus (dominante o recesivo). Se suele configurar típicamente entre el 75% y el 87.5% de la duración total del bit.

Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time)

En el protocolo CAN, la resincronización continua de los nodos receptores se realiza exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$).

Caso 1: Flanco Ideal (Sin Ajuste)



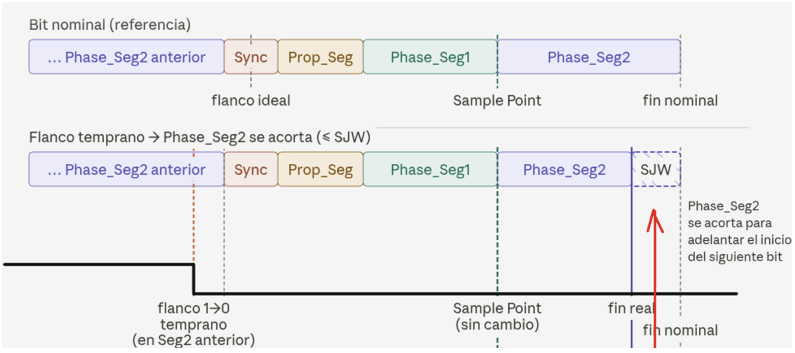
- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega exactamente dentro del segmento `Sync_Seg` (de duración fija $1t_q$).
- **Estado de Reloj:** El oscilador del nodo receptor está perfectamente sincronizado con el del transmisor.
- **Acción:** No se realiza ningún ajuste en la duración de los segmentos ($\Delta t_q = 0$) y el **Sample Point** permanece en su posición nominal.

Caso 2: Flanco Tardío (Reloj del Transmisor Más Lento)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega con retraso, cayendo en los segmentos `Prop_Seg` o `Phase_Seg1` en lugar de `Sync_Seg`.
- **Estado de Reloj:** El reloj del nodo transmisor es **más lento** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **alarga** el segmento `Phase_Seg1` agregando ciclos t_q adicionales.
  - La cantidad de ciclos agregados es igual al error de fase detectado, acotado por el valor máximo de `SJW` ($+\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Se desplaza el **Sample Point** hacia la derecha (retrasando el momento de muestreo) para capturar correctamente el dato.

Caso 3: Flanco Temprano (Reloj del Transmisor Más Rápido)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega antes de lo esperado, cayendo dentro del segmento `Phase_Seg2` del bit anterior.
- **Estado de Reloj:** El reloj del nodo transmisor es **más rápido** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **acorta** el segmento `Phase_Seg2` substrayendo ciclos t_q de su duración.
  - La reducción está acotada por el valor máximo configurado en `SJW` ($-\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Adelanta la finalización del bit actual e inicia el próximo ciclo antes de lo nominal para reengancharse con el transmisor rápido.

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 1: Ideal	Dentro de `Sync_Seg`	Sincronizado	Sin ajuste ($\Delta t_q = 0$).
Caso 2: Tardío	En `Prop_Seg` / `Phase_Seg1`	Más lento	Se alarga `Phase_Seg1` ($+\Delta t_q \leq \text{SJW}$).
Caso 3: Temprano	En `Phase_Seg2` anterior	Más rápido	Se acorta `Phase_Seg2` ($-\Delta t_q \leq \text{SJW}$).

> ⚠️ **¡Sincronización Solo en Flancos Descendentes!**
>
> En CAN, la sincronización se realiza **exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$, paso a dominante)**.
>
> Los flancos ascendentes ($0 \rightarrow 1$, paso a recesivo) **no se utilizan** debido a que la subida del bus es pasiva (depende de la resistencia **pull-middle** y capacitancias parásitas del cable), generando pendientes lentas e imprecisas.

Filtrado por Hardware y Manejo de Errores

1. 1. 6. 1 Filtrado de Mensajes (Filter y Mask)

Para evitar sobrecargar la CPU del microcontrolador analizando mensajes no relevantes, el módulo controlador CAN (como el FlexCAN en el Kinetis K64F) filtra las tramas por hardware mediante dos registros:

- **Filter (Identificador de Filtro):** Contiene el patrón binario de ID específico que el nodo está programado para recibir.
- **Mask (Máscara de Aceptación):** Define qué bits del ID deben coincidir obligatoriamente:
  - Bit de máscara en `'1'`: **Verificación estricta** (el bit del ID entrante debe ser idéntico al bit del filtro).
  - Bit de máscara en `'0'`: **Don't Care** (se ignora la coincidencia de ese bit en particular).

1. 1. 6. 2 Estados del Nodo y Manejo de Errores

Cada nodo CAN posee dos contadores internos de error mantenidos por hardware: `TEC` (**Transmit Error Counter**) y `REC` (**Receive Error Counter**). Según sus valores, el nodo transiciona entre tres estados de operación:

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Active	$\text{TEC} < 128$ y $\text{REC} < 128$	Operación normal. Al detectar un error, transmite un **Active Error Flag** (6 bits dominantes), abortando la trama de toda la red.
Error Passive	$\text{TEC} \geq 128$ o $\text{REC} \geq 128$	El nodo está degradado. Transmite un **Passive Error Flag** (6 bits recesivos), evitando interrumpir la comunicación de otros nodos.
Bus Off	$\text{TEC} > 255$	El nodo se desconecta lógicamente del bus. No puede transmitir ni recibir hasta reiniciar el módulo FlexCAN.

Clase 8

Periféricos de Temporización y Módulo FlexTimer (FTM)

Fundamentos y Necesidad de Hardware Dedicado

En sistemas embebidos en tiempo real, el uso de interrupciones periódicas por software para generar o medir pulsos presenta limitaciones críticas cuando los anchos de pulso son muy angostos o cuando se requiere una precisión estricta sin sufrir la latencia de atención de la CPU.

Por esta razón, los microcontroladores incorporan bloques periféricos de **Timer por hardware**. Los dos conceptos fundamentales sobre los que se construyen estos periféricos son:

- **Output Compare (OC):** Permite programar un cambio de estado automático en un pin de salida (Toggle, Set, Clear) cuando el contador alcanza un valor predeterminado en el registro de comparación. Este evento ocurre por hardware de forma determinística, eliminando el impacto del jitter e impredecibilidad causados por la latencia de la ISR.

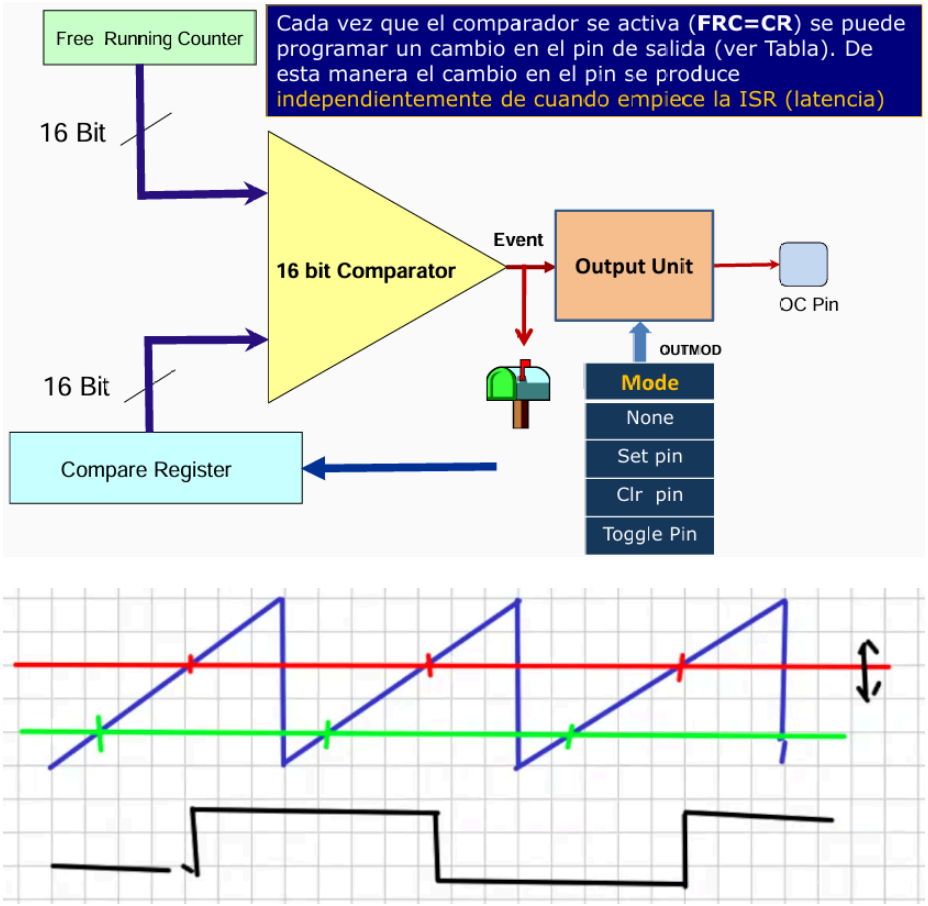


Figura 9: Ejemplo de uso con duty cycle constante

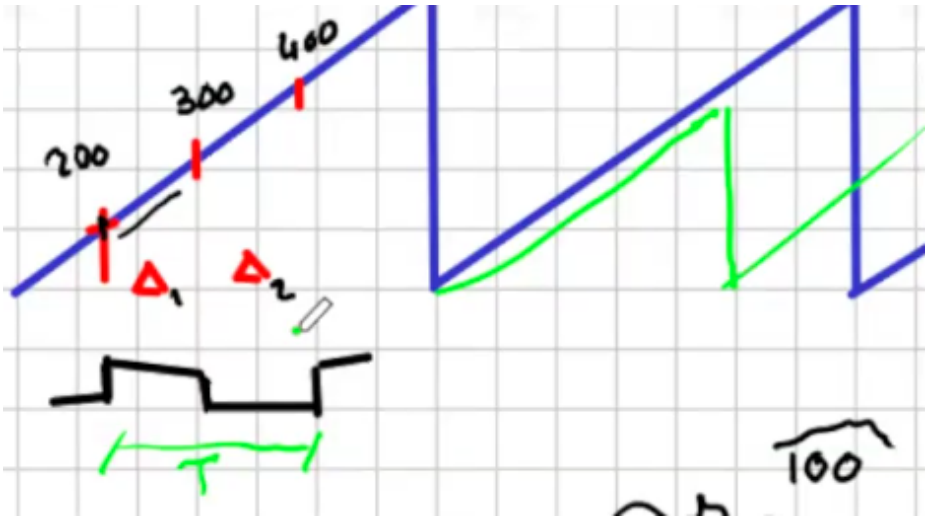


Figura 10: Ejemplo de uso con duty cycle utilizando deltas

- **Input Capture (IC):** Registra (**latch**) el valor instantáneo del contador principal ante la llegada de un flanco en un pin de entrada. Se utiliza para medir frecuencias, períodos o anchos de pulso extremadamente angostos sin sobrecargar la CPU.

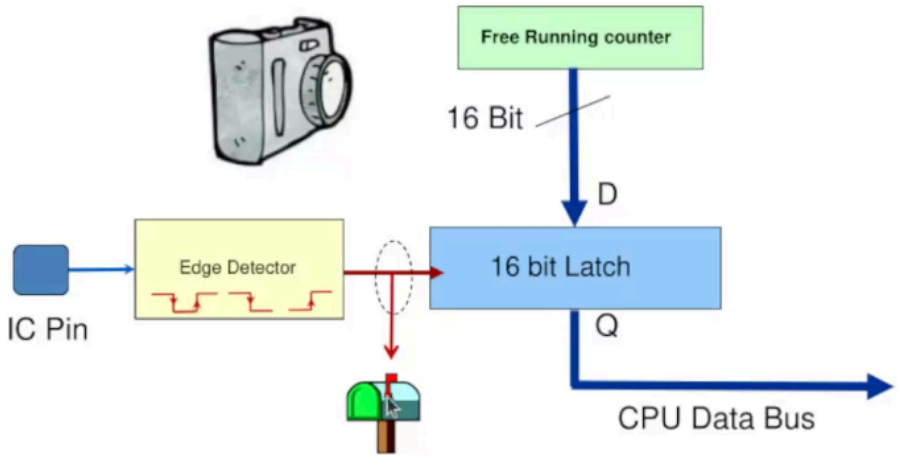


Figura 11: 'Se le saca una foto al contador en el instante del flanco de entrada'

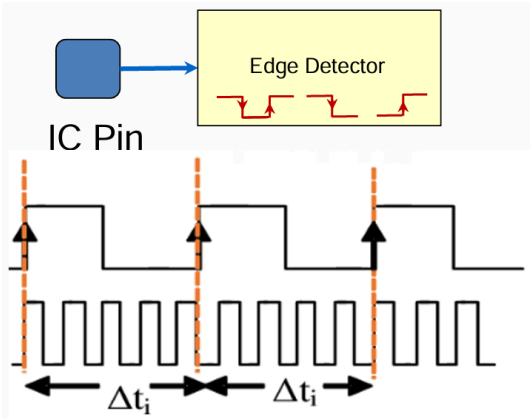


Figura 12: t_i corresponde al clock de FRC (Free Running Counter).)

> 💡 **Principio de Funcionamiento por Hardware**
>
> Ambas técnicas se basan en un contador principal de 16 bits (**Free Running Counter / FTM Counter**) accionado por un reloj del sistema [1, 11, 12]. Tanto el latch de captura como la conmutación de los pines de salida se realizan en el mismo ciclo de reloj del evento físico.

Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F

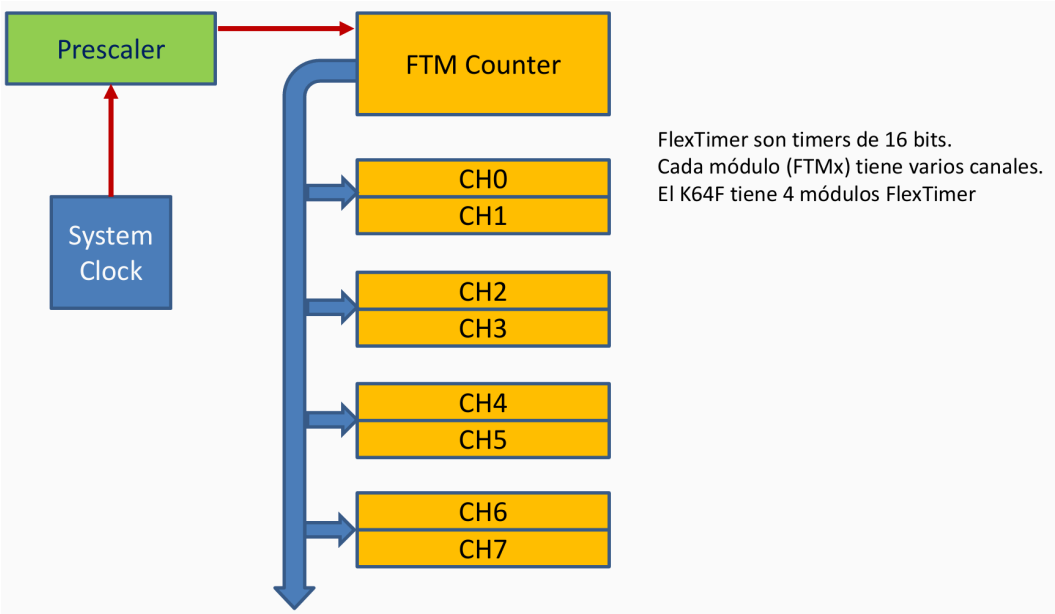


Figura 13: FTM counter y sus-submodulos.

El **FlexTimer Module (FTM)** es un temporizador multicanal de 16 bits integrado en la subfamilia Kinetis K64 (microcontrolador `MK64FN1M0VLL12` montado en la placa FRDM-K64F).

2. 1. 2. 1 Instancias y Distribución de Canales

El microcontrolador K64F dispone de 4 instancias físicas del módulo FTM:

Instancia FTM	N° de Canales	Funciones Principales / Usos
FTM0	8 canales (CH0..CH7)	Control de motores trifásicos, PWM general, motores paso a paso.
FTM1	2 canales (CH0..CH1)	Decodificador en cuadratura (**Quadrature Decoder**) o propósito general.
FTM2	2 canales (CH0..CH1)	Decodificador en cuadratura o propósito general.
FTM3	8 canales (CH0..CH7)	Control de motores trifásicos, PWM y medición de señales.

> ⚙️ **Habilitación del Reloj (Clock Gating)**
>
> Antes de acceder a los registros de cualquier módulo FTM, es obligatorio activar la señal de reloj (**Clock Gate**) en el módulo SIM (**System Integration Module**):
> - **FTM0, FTM1, FTM2:** Se habilitan mediante los bits `SIM_SCGC6_FTM0_MASK`, `SIM_SCGC6_FTM1_MASK`, `SIM_SCGC6_FTM2_MASK` en `SIM->SCGC6`.
> - **FTM2, FTM3:** Se habilitan mediante los bits `SIM_SCGC3_FTM2_MASK` y `SIM_SCGC3_FTM3_MASK` en `SIM->SCGC3`.

2. 1. 2. 2 Fuentes de Reloj (Clock Source) y Prescaler

El contador del FTM recibe su señal de reloj desde una llave de selección configurable en el registro de control `FTMx_SC` mediante el campo `CLKS[1:0]`:

- `00`: Sin reloj seleccionado (deshabilita el contador FTM).
- `01`: **System Clock:** Corresponde internamente al **Bus Clock** del microcontrolador ($f_{\text{bus}} = 50$ MHz en la placa FRDM-K64F a 100 MHz de Core).
- `10`: **Fixed Frequency Clock (`MCGFFCLK`):** Reloj de frecuencia fija independiente.
- `11`: **External Clock (`FTM_CLKINx`):** Reloj desde pines externos seleccionados vía `SIM_SOPT4`.

El valor del reloj seleccionado se divide posteriormente mediante un **Prescaler (PS)** de 3 bits (`PS[2:0]` en `FTMx_SC`), el cual soporta factores de división entre 1 y 128 (2^n con $n = 0..7$):

$$f_{\text{FTM_CNT}} = \frac{f_{\text{Clock_Source}}}{\text{Prescaler}}$$

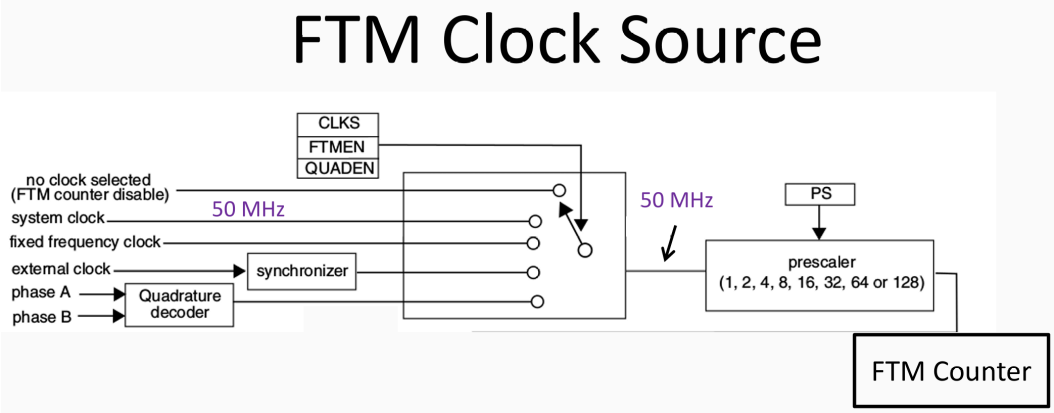


Figura 14: Clock Source

🛠️ Cálculo de Frecuencia y Resolución de Conteo

Si se selecciona el **Bus Clock** (50 MHz) y se configura un prescaler de 4 (`FTM_PSC_x4`): $f_{\text{FTM_CNT}} = \frac{50\text{ MHz}}{4} = 12.5\text{ MHz}$ Resolución de conteo (período de tick) $= \frac{1}{12.5\text{ MHz}} = 80$ ns

```
void FTM_SetPrescaler (FTM_t ftm, FTM_Prescal_t data)
{
    ftm->SC = (ftm->SC & ~FTM_SC_PS_MASK) | FTM_SC_PS(data);
}
```

En MK64F12.h

```
#define FTM_SC_PS(x)        (((uint32_t)(((uint32_t)(x))<<FTM_SC_PS_SHIFT))&FTM_SC_PS_MASK)
#define FTM_SC_PS_MASK      0x7u
#define FTM_SC_PS_SHIFT     0
```

```
typedef enum
{
FTM_PSC_x1= 0x00,
FTM_PSC_x2= 0x01,
FTM_PSC_x4= 0x02,
FTM_PSC_x8= 0x03,
FTM_PSC_x16= 0x04,
FTM_PSC_x32= 0x05,
FTM_PSC_x64= 0x06,
FTM_PSC_x128= 0x07,
} FTM_Prescal_t;
```

Ejemplo:

```
FTM_SetPrescaler(FTM0, FTM_PSC_x4); // BusClock/2=Sysclk/2= 50MHz y presccaler = divx4
```

Entonces: FTM_Coutner_Clk= 50MHz /4= 12.5MHz (Resolución 80 ns)

2. 1. 2. 3 Modos de Conteo del Contador (`FTM Counter`)

El FTM soporta tres modos de conteo configurables mediante los registros `FTMx_SC` y `FTMx_QDCTRL`:

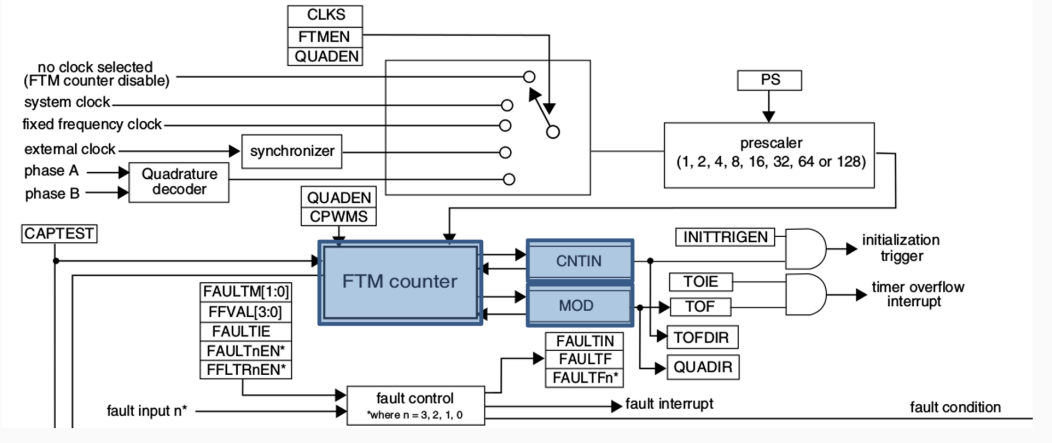


Figura 16: FTM: Up Counting

1. **Up Counting (Conteo Ascendente):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 0`.
   - **Comportamiento:** El contador se carga con `CNTIN` y se incrementa en cada tick hasta alcanzar `MOD`. Al llegar a `MOD`, se recarga automáticamente con `CNTIN` e impone `TOF = 1` (Timer Overflow Flag).
   - **Período de conteo:**

$$\text{Período} = (\text{MOD} - \text{CNTIN} + 1) \times t_{\text{FTM_CNT}}$$

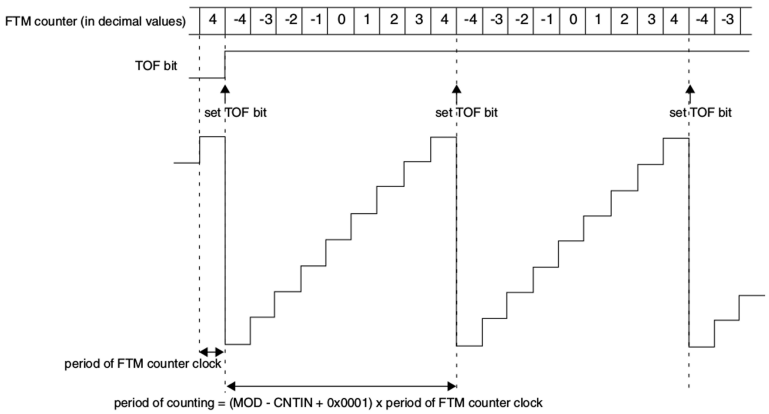


Figura 17: Up Counting, cuenta la cantidad de overflows.

2. **Up-Down Counting (Conteo Ascendente/Descendente - Center Aligned):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 1`.
   - **Comportamiento:** Cuenta desde `CNTIN` hasta `MOD`, y al llegar a `MOD` comienza a decrementar hasta regresar a `CNTIN`. El flag `TOF = 1` se activa al pasar de `MOD` a `MOD - 1`.
   - **Período de conteo:**

$$\text{Período} = 2 \times (\text{MOD} - \text{CNTIN}) \times t_{\text{FTM_CNT}}$$

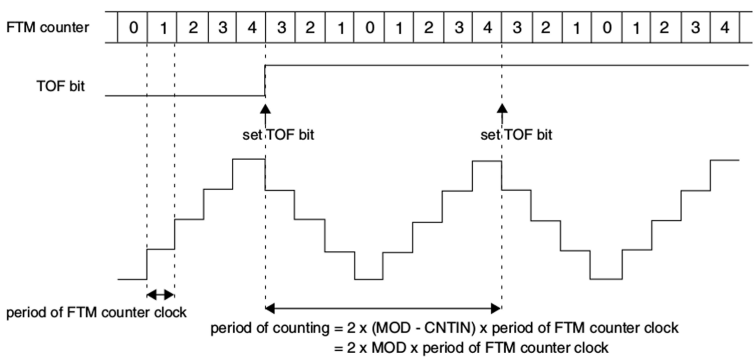


Figura 18: Up-Down Counting

3. **Free Running Counter:**

- **Comportamiento:** Se obtiene fijando `CNTIN = 0x0000` y `MOD = 0xFFFF`. El contador recorre todo el rango de 16 bits ($0..65535$) continuamente. `TOF = 1` cuando el contador pasa de 0xFFFF to 0x0000

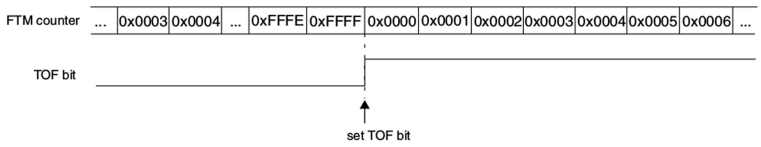


Figura 19: Free Running Counter

> ⚙️ **Registro de Filtro de Overflows (NUMTOF)**
>
> El FTM incluye el registro `NUMTOF[4:0]`, el cual especifica la cantidad de veces que el contador debe sufrir un overflow antes de que el hardware active efectivamente el bit de interrupción `TOF = 1`. Esto permite extender los intervalos de interrupción sin sobrecargar la CPU.

Modos de Operación y Configuración por Canal

2. 1. 3. 1 Modo Compatibilidad TPM (`FTMEN = 0`) vs. Modo FTM Avanzado (`FTMEN = 1`)

El módulo FTM incluye dos modos globales de operación definidos en el registro `FTMx_MODE`:

- **FTMEN = 0 (TPM Legacy):** Modulo compatible con los timers TPM antiguos (arquitectura HCS08). Solo permite el uso del conjunto básico de registros.
- **FTMEN = 1 (FTM Advanced Functions):** Habilita todas las funciones avanzadas: Combine Mode, Deadtime Insertion, Fault Control, Quadrature Decoder, PWM Synchronization e Inversión de Salidas.

> ⚠️ **¡Protección de Escritura (WPDIS)!**
>
> Para escribir en el bit `FTMEN` o en registros avanzados, se debe desactivar primero la protección contra escritura poniendo en `'1'` el bit `WPDIS` (`Write Protection Disable`) en `FTMx_MODE`.

2. 1. 3. 2 Modos de Trabajo Individuales y Combinados

Modo de Canal	Configuración de Bits	Descripción / Comportamiento

Input Capture (IC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 00`
	`ELSnB:ELSnA = 01` (Ascendente)
	`ELSnB:ELSnA = 10` (Descendente)
	`ELSnB:ELSnA = 11` (Ambos flancos)
		Guarda el valor instantáneo del contador `FTM_CNT` en `FTMx_CnV` al detectar el flanco configurado en el pin. Genera interrupción `CHnF = 1`.

Output Compare (OC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 01`
	`ELSnB:ELSnA = 01` (Toggle)
	`ELSnB:ELSnA = 10` (Clear)
	`ELSnB:ELSnA = 11` (Set)
		Compara `FTM_CNT` con `FTMx_CnV`. Al coincidir (`FTM_CNT == CnV`), conmuta la salida del pin automáticamente y activa `CHnF = 1`.

Edge-Aligned PWM (EPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 1X`
	`ELSnB:ELSnA = 10` (High-true)
	`ELSnB:ELSnA = X1` (Low-true)
		PWM alineado al borde con conteo ascendente. La salida cambia al inicio del período y al coincidir con `CnV`.
		Ancho de pulso:
		$$\mathrm{CnV} - \mathrm{CNTIN}$$
		.

Center-Aligned PWM (CPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=1` `ELSnB:ELSnA = 10` (High-true)	PWM centrado con conteo Up-Down. **Ancho de pulso:**

Modo de Canal	Configuración de Bits	Descripción / Comportamiento
		$2 \times (\text{CnV} - \text{CNTIN})$. **Período:** $2 \times (\text{MOD} - \text{CNTIN})$.
Combine PWM Mode	`FTMEN=1`, `COMBINE=1`, `CPWMS=0` Requiere un par de canales (n par y $n+1$).	El canal n define el borde de encendido (`CnV`) y el canal $n+1$ el borde de apagado (`C(n+1)V`). **Ancho de pulso:** $\lvert \text{C}(n+1)\text{V} - \text{CnV} \rvert$.

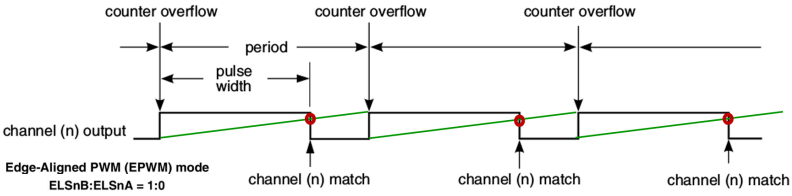


Figura 20: EPWM

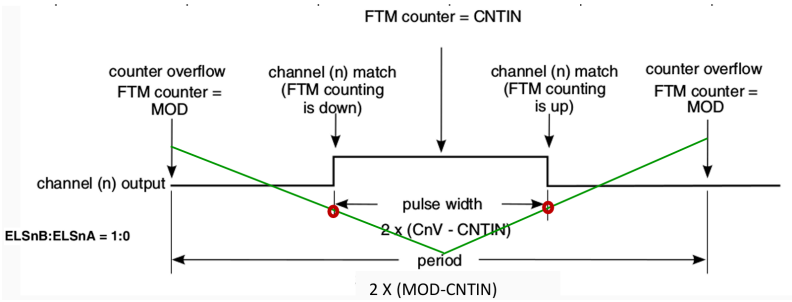


Figura 21: CPWM

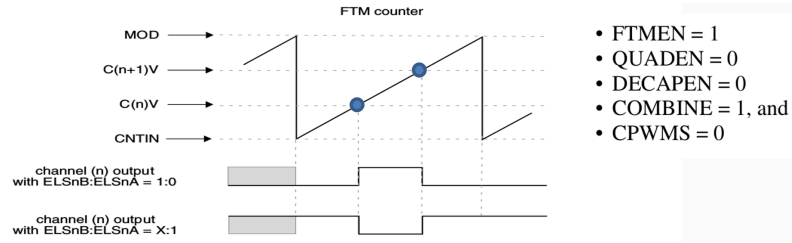


Figura 22: Combine PWM

2. 1. 3. 3 Filtrado Digital en Captura de Entrada (Input Capture Filter)

El FTM incluye filtros digitales por hardware en los canales de entrada $0, 1, 2, 3$ para suprimir ruido o rebotes de señal.

El filtro exige que la señal permanezca estable durante una ventana temporal configurable en `FTMx_FILTER` mediante el campo `CHnFVAL[3:0]`:

$\text{Ventana de estabilidad} = 4 + 4 \times \text{CHnFVAL}[3{:}0]$ (en ciclos de clock del sistema)

2. 1. 3. 4 Inserción de Tiempo Muerto (Deadtime Insertion)

En aplicaciones de electrónica de potencia (como el control de motores DC mediante puentes H o inversores trifásicos con MOSFETs/IGBTs), los dos conmutadores de una misma rama nunca deben conducir simultáneamente para evitar un cortocircuito directo (**shoot-through**) de la fuente de alimentación.

El FTM permite configurar salidas complementarias entre los canales n y $n+1$ (`COMP = 1` en `COMBINE`) e insertar de forma automática un tiempo muerto (**Deadtime**) a través del registro `FTMx_DEADTIME`:

> ⚙️ **Registro FTMx_DEADTIME**
>
> - `DTPS[1:0]:` Prescaler del reloj de tiempo muerto (divisores $1, 4, 16$).
> - `DTVAL[5:0]:` Número de ciclos de reloj del prescaler asignados como retardo de conmutación.

2. 1. 3. 5 Control de Falla por Hardware (Fault Control)

El FTM monitorea hasta 4 entradas físicas de falla (`FAULTIN0..3`) para proteger las etapas de potencia. Ante una condición de sobrecorriente o sobretemperatura detectada por hardware, el módulo deshabilita inmediatamente las salidas PWM y las fuerza a un estado seguro sin esperar intervención del software.

Fuentes de Interrupción y Vectores NVIC

Cada módulo FTM consolida sus múltiples eventos de interrupción en **un único vector de interrupción** en el controlador NVIC del microcontrolador:

Fuente de Interrupción	Flags de Habilitación	Registro / Ubicación
Timer Overflow Interrupt	`TOIE = 1` y `TOF = 1`	Registro `FTMx_SC`.
Channel Interrupt (n)	`CHnIE = 1` y `CHnF = 1`	Registro de canal `FTMx_CnSC`.
Fault Interrupt	`FAULTIE = 1` y `FAULTF = 1`	Registro `FTMx_MODE`.

> ⚙️ **Vectores IRQ en el Vector Table de K64F**
>
> - `FTM0_IRQHandler` → Vector 42 en NVIC (`FTM0_IRQn`).
> - `FTM1_IRQHandler` → Vector 43 en NVIC (`FTM1_IRQn`).
> - `FTM2_IRQHandler` → Vector 44 en NVIC (`FTM2_IRQn`).
> - `FTM3_IRQHandler` → Vector 45 en NVIC (`FTM3_IRQn`).

Clase 9

ADC: Conversor Analógico–Digital

Arquitectura y Señales de Entrada

El Kinetis K64F incorpora **dos módulos ADC** de 16 bits con arquitectura **SAR (Successive Approximation Register)**. Cada instancia admite:

- **Hasta 24 canales Single Ended** (`AD[y]`, donde `y` es el número de canal).
- **4 canales Diferenciales** (`DAD[x][y]`, con `x = P(+)` o `M(−)`).

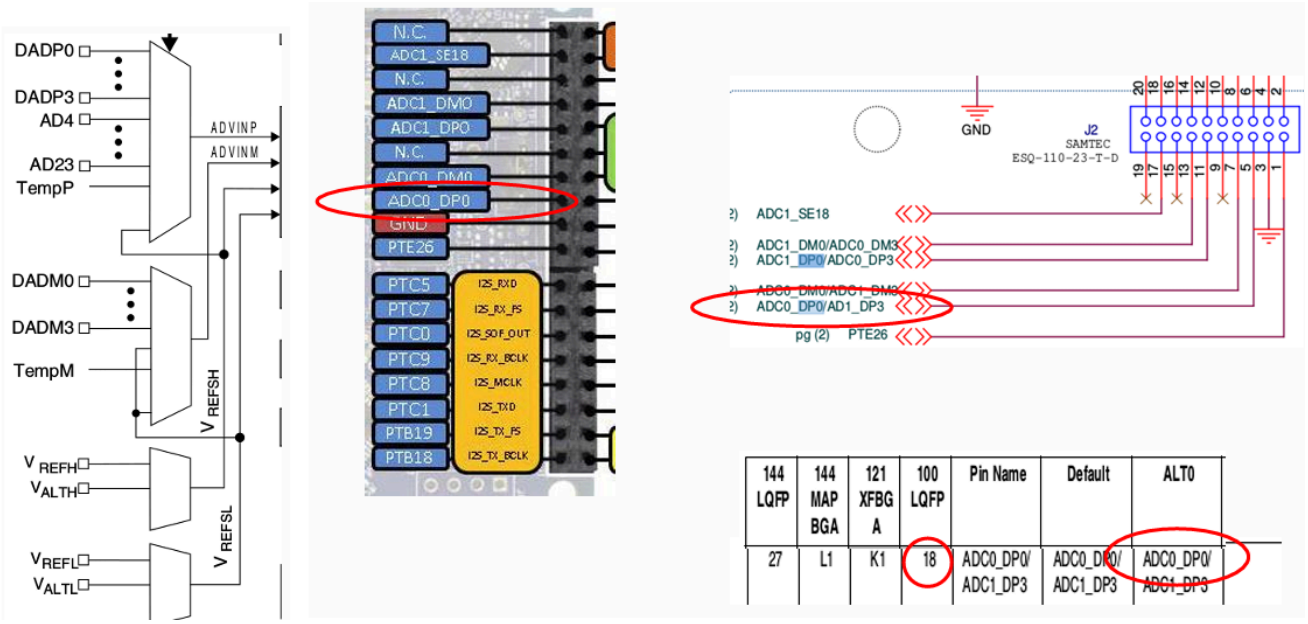


Figura 23: El canal a convertir se selecciona con el campo `ADCH[4:0]` del registro `ADCx_SC1`

> 💡 **Notación de Canales ADC**
>
> - **Diferencial:** `DAD[x][y]` donde `x = M(−)` o `P(+)`, `y = N° de canal`.
> - **Single Ended:** `AD[y]` — equivalente a `DADP[y]` cuando `DIFF = 0`.
>
> Al configurar `DIFF = 0`, el pin `DADP[y]` se reasigna como entrada Single Ended `AD[y]`.

Selección de Canal e Inicio de Conversión

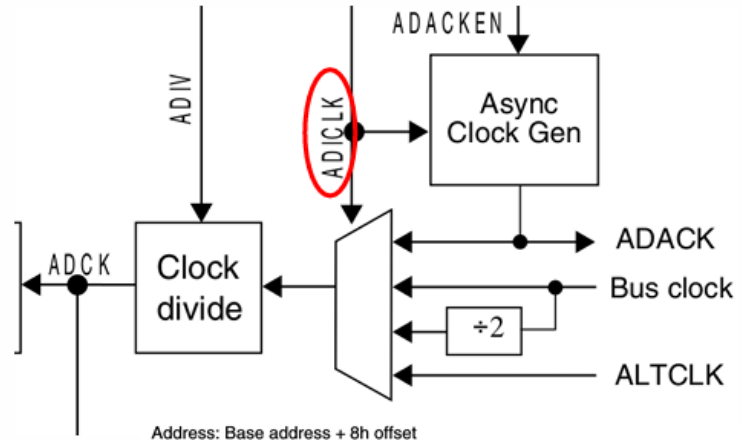
El canal a convertir y el modo de operación se configuran en el registro `ADCx_SC1[0]` mediante el campo `ADCH[4:0]`. Escribir en este registro **inicia la conversión**.

> ⚙️ **Registro ADCx_SC1 — Status and Control 1**
>
> - `ADCH[4:0]`: Selección del canal de entrada. Escribir en este campo inicia la conversión.
> - `DIFF`: `0` = modo Single Ended; `1` = modo Diferencial.
> - `AIEN`: `1` = habilita la interrupción al finalizar la conversión (`COCO = 1`).
> - `COCO` (read-only): **Conversion Complete Flag**. Se activa cuando la conversión finaliza. Se limpia al leer el registro `ADCx_Rn`.

```
// Inicio de conversión por software (polling o con interrupción)
void ADC_Start(ADC_t adc, ADCChannel_t channel, ADCMux_t mux)
```

```
{
    adc->CFG2 = (adc->CFG2 & ~ADC_CFG2_MUXSEL_MASK) | ADC_CFG2_MUXSEL(mux);
    adc->SC1[0] = ADC_SC1_AIEN(ADC_interrupt[x]) | ADC_SC1_ADCH(channel);
}
```

Reloj del ADC



El módulo ADC requiere una señal de reloj de entrada. Dicha señal se configura en `ADCx_CFG1` mediante dos campos:

Campo	Registro	Descripción
`ADICLK[1:0]`	`ADCx_CFG1[1:0]`	Selector de fuente de reloj: `00` = Bus Clock; `01` = Bus Clock/2; `10` = Alternate Clock (`ALTCLK`); `11` = ADACK (ADC Asynchronous Clock).
`ADIV[1:0]`	`ADCx_CFG1[6:5]`	Prescaler del reloj ADC. Factores de división: `00`=1; `01`=2; `10`=4; `11`=8.

```
ADC0->CFG1 = ADC_CFG1_ADICLK(0x00);  // Bus Clock como fuente
ADC0->CFG1 = ADC_CFG1_ADIV(0x00);    // Sin prescaler (dividir por 1)
```

Resolución del ADC

La resolución de conversión se configura en el campo `MODE[1:0]` del registro `ADCx_CFG1[3:2]`:

MODE[1:0]	DIFF = 0 (Single Ended)	DIFF = 1 (Diferencial, 2's Comp.)
`00`	8 bits	9 bits
`01`	12 bits	13 bits
`10`	10 bits	11 bits
`11`	16 bits	16 bits

Lectura del Resultado y Flag EOC

Una vez completada la conversión, el resultado se almacena en `ADCx_R[0]` y el flag `COCO` se pone en `1`.

```
// Verificar si la conversión finalizó (EOC - End Of Conversion)
bool ADC_IsReady(ADC_t adc)
{
    return adc->SC1[0] & ADC_SC1_COCO_MASK;
}
```

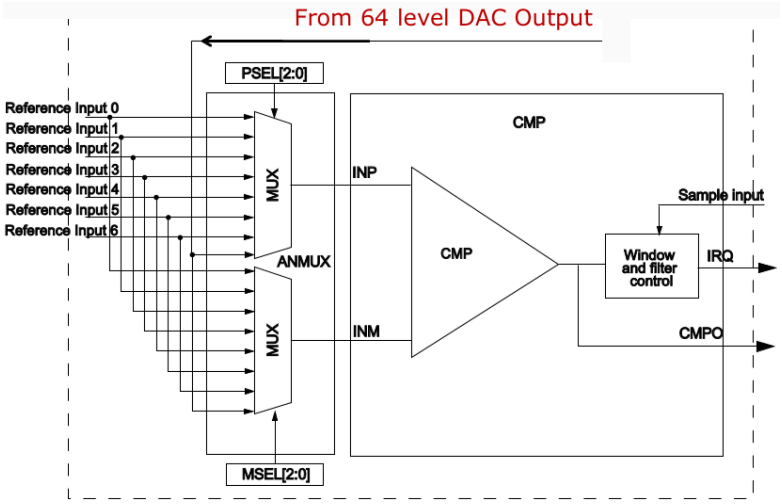
```
// Leer el resultado de la conversión (lectura limpia COCO)
ADCData_t ADC_getData(ADC_t adc)
{
    return adc->R[0];
}
```

> ⚠️ **Limpieza del Flag COCO**
>
> El flag `COCO` se limpia automáticamente al **leer** el registro `ADCx_Rn`. No es necesario limpiarlo de forma explícita.

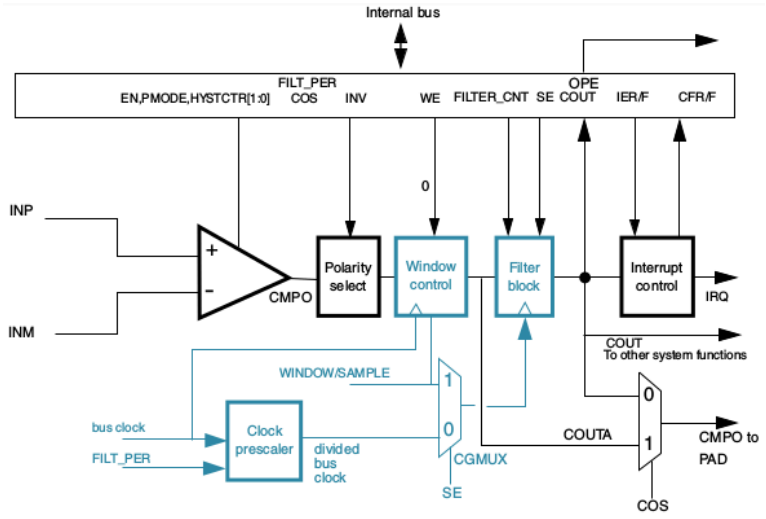
Comparador Analógico (CMP)

El K64F incorpora un **Comparador Analógico (CMP)** integrado que puede usarse en conjunto con el ADC. Sus características principales son:

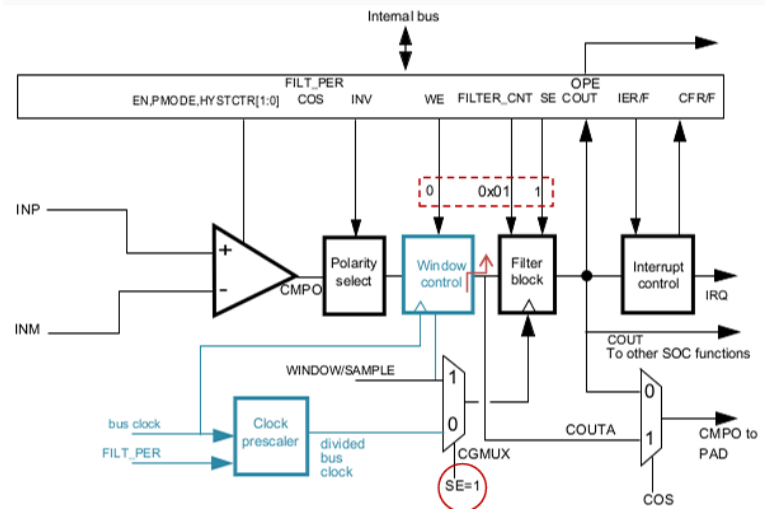
- **DAC interno de 64 niveles** para generar la tensión de referencia de comparación.



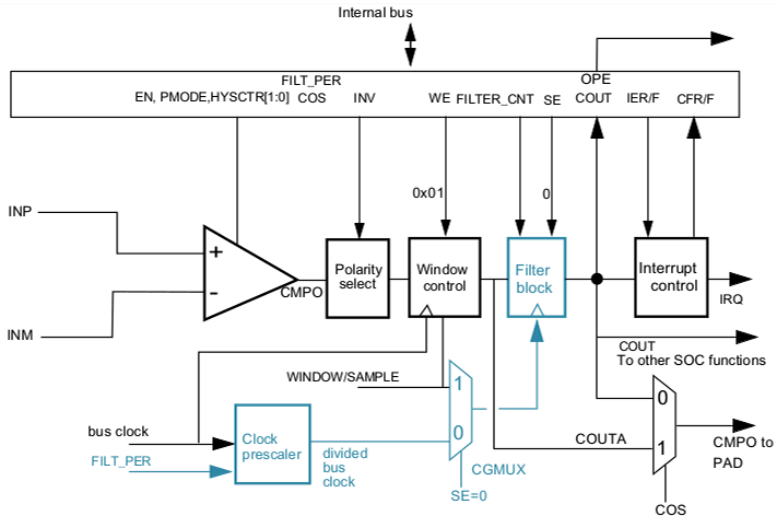
- **Control de histéresis** configurable por hardware para evitar oscilaciones en la salida.
- Múltiples modos de operación:
  - **Continuo:** La salida `COUT` refleja directamente la comparación en modo combinacional (sin reloj).

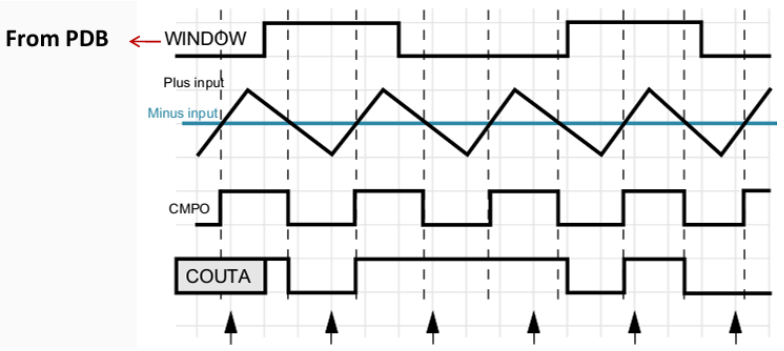


  - **Sampled Non-Filtered:** `COUTA` se muestrea en cada flanco ascendente del reloj del bloque de filtro.

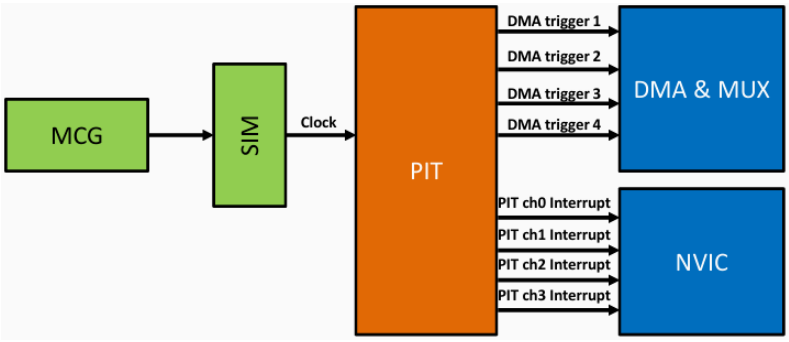


- **Windowed:** La salida analógica solo pasa a `COUTA` cuando la señal `WINDOW` está en alto (típicamente proveniente del PDB). El último valor queda latched cuando `WINDOW = 0`.





PIT: Periodic Interrupt Timer



Descripción General y Uso

El módulo **PIT (Periodic Interrupt Timer)** consiste en un conjunto de **4 timers de conteo descendente de 32 bits** cuyas funciones principales son:

- **Generar interrupciones periódicas** a intervalos configurables.
- **Proveer triggers periódicos** para los módulos DMA y ADC.

El PIT puede disparar transferencias DMA (a través del DMAMUX) y disparar conversiones ADC (a través de `SIM_SOPT7`), permitiendo adquisición y transferencia de datos completamente por hardware sin intervención de la CPU.

Clock y Habilitación

El **reloj del PIT es el Bus Clock** ($f_{\text{bus}} = 50$ MHz en la FRDM-K64F):

```
SIM->SCGC6 |= SIM_SCGC6_PIT_MASK;  // Habilitar clock gate del PIT
PIT->MCR = 0x00;                    // Encender el módulo PIT (MDIS = 0)
```

Registros Principales

Registro	Descripción
`PIT_MCR`	Module Control Register. `MDIS = 0`: módulo activo. `FRZ = 1`: detiene timers en modo debug.
`PIT_LDVALn`	Timer Load Value Register. Define el período: el timer cuenta desde `LDVALn` hasta `0` y genera la interrupción/trigger.
`PIT_CVALn`	Current Timer Value Register (solo lectura). Valor actual del contador descendente.
`PIT_TCTRLn`	Timer Control Register. `TEN = 1`: habilita el timer. `TIE = 1`: habilita la interrupción. `CHN = 1`: encadena el timer n al timer $n-1$ (Chain Mode).
`PIT_TFLGn`	Timer Flag Register. `TIF = 1`: indica que el timer expiró. Se limpia escribiendo `1` en este bit.

> 💡 **Cálculo del Período del PIT**
>
> Con $f_{\text{bus}} = 50$ MHz (período $t_{\text{bus}} = 20$ ns):
>
> $\text{LDVALn} = \frac{T_{\text{deseado}}}{t_{\text{bus}}} - 1$
>
> **Ejemplo:** Para $T = 5.12$ ms: $\text{LDVAL} = \frac{5.12 \times 10^{-3}}{20 \times 10^{-9}} - 1 = 256000 - 1$

Modo Chain (Timers Encadenados)

El **Chain Mode** permite extender el rango de temporización concatenando dos timers. El timer n solo decrementa cuando el timer $n-1$ llega a cero:

🛠️ Ejemplo: Timer de 1 Minuto con Chain Mode

Con $f_{\text{bus}} = 100\ \text{MHz}$ ($t_{\text{bus}} = 10\ \text{ns}$), para generar un período de 1 minuto ($60\ \text{s} = 6\ \text{s} \times 10$):

- **Timer 1:** $T = 6\ \text{s} \rightarrow \text{LDVAL1} = 6 \times 10^9\ \text{ns}\ /10\ \text{ns} - 1 = 6 \times 10^8 - 1$
- **Timer 2:** 10 ciclos del Timer 1 $\rightarrow \text{LDVAL2} = 10 - 1 = 9$ (con `CHN = 1`)

```
PIT->MCR = 0x00;
// Timer 2 (encadenado al Timer 1)
PIT->CHANNEL[2].LDVAL = 10 - 1;
PIT->CHANNEL[2].TCTRL = PIT_TCTRL_TIE_MASK | PIT_TCTRL_CHN_MASK | PIT_TCTRL_TEN_MASK;
// Timer 1 (fuente de clock para Timer 2)
PIT->CHANNEL[1].LDVAL = 600000000 - 1;
PIT->CHANNEL[1].TCTRL = PIT_TCTRL_TEN_MASK;
```

PIT como Trigger para DMA y ADC

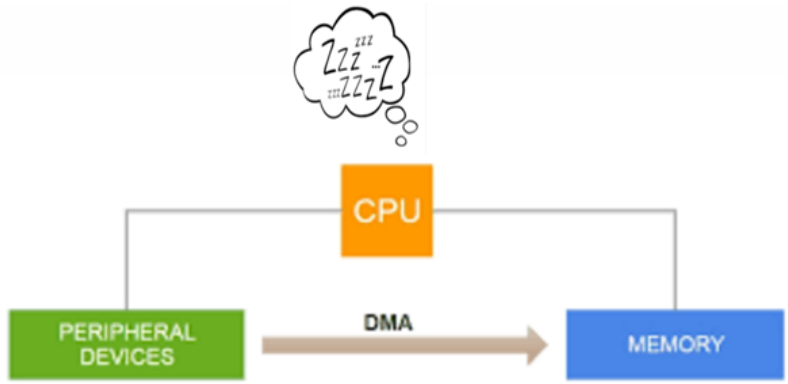
⚙️ PIT → DMA (DMAMUX)

Cada canal PIT (0..3) puede actuar como trigger periódico para el DMA MUX. Se configura en el DMAMUX activando `TRIG = 1` en `DMAMUX_CHCFGn`.

⚙️ PIT → ADC (SIM_SOPT7)

El PIT puede disparar conversiones ADC directamente. Se configura en `SIM_SOPT7` seleccionando el trigger y habilitando `ADCxALTTRGEN`.

DMA: Direct Memory Access



Concepto y Motivación

El **DMA (Direct Memory Access)** es un controlador de hardware que realiza transferencias de datos entre memoria y periféricos (o entre dos zonas de memoria) **sin intervención de la CPU**. Esto libera al procesador para ejecutar otras tareas mientras el DMA opera de forma autónoma sobre el bus del sistema.

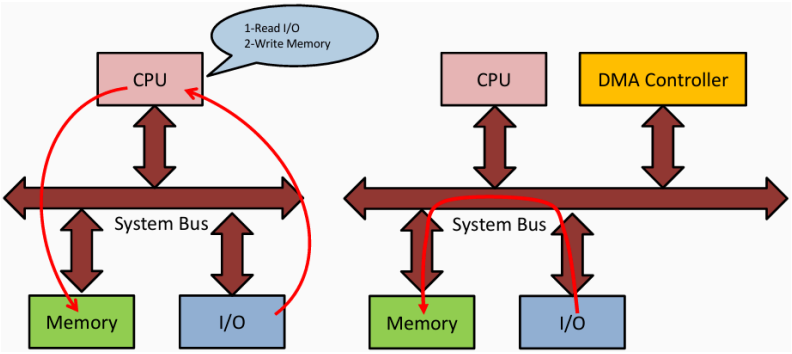
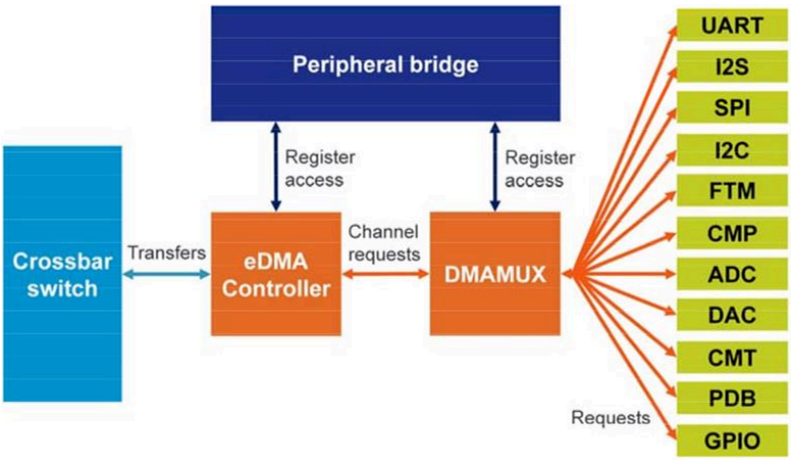


Figura 32: Programmed I/O Transfer vs **DMA Transfer**

El flujo de una transferencia DMA sigue estos pasos:

1. **Configuración:** La CPU programa los registros del controlador DMA (dirección origen, destino y cantidad de datos).
2. **Solicitud del Periférico:** El periférico genera un request de DMA cuando está listo para transferir datos.
3. **Hold Request:** El controlador DMA solicita el bus a la CPU.
4. **Hold Acknowledge:** La CPU libera el bus y pone sus salidas en alta impedancia (HI-Z).
5. **Transferencia:** El controlador DMA realiza la transferencia de datos.
6. **Interrupción Final:** Al completar la transferencia, el DMA genera una interrupción hacia la CPU.

Arquitectura del eDMA en el K64F



El Kinetis K64F incorpora un módulo **eDMA (Enhanced DMA)** de **16 canales** con las siguientes características:

- **DMAMUX (DMA Multiplexer):** Rutea hasta **52 fuentes de DMA** (slots de periféricos) hacia los 16 canales del eDMA.

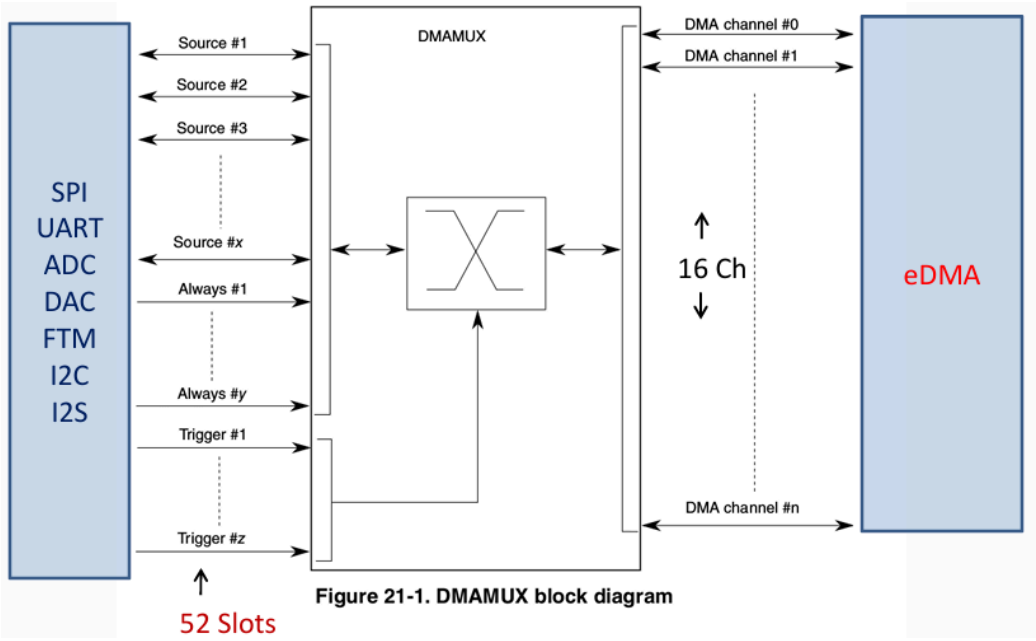


Figure 21-1. DMAMUX block diagram

- **TCD Memory:** Cada canal del eDMA tiene un **TCD (Transfer Control Descriptor)** de 32 bytes (8 palabras de 32 bits) que describe completamente la transferencia.

DMAMUX: Modos de Operación

El DMAMUX tiene tres modos configurables para cada canal mediante el registro `DMAMUX_CHCFGn`:

Modo	ENBL / TRIG	Comportamiento
Disabled	`ENBL=0`	Canal deshabilitado. Usar para reconfigurar el DMAMUX antes de cambiar la fuente.
Normal	`ENBL=1`, `TRIG=0`	La fuente del periférico (slot) se rutea directamente al canal DMA. La transferencia se inicia cuando el periférico genera un request.
Periodic Trigger	`ENBL=1`, `TRIG=1`	El request del periférico queda bloqueado (**gated**) hasta que el PIT genera un trigger. Solo los canales 0..3 soportan este modo.

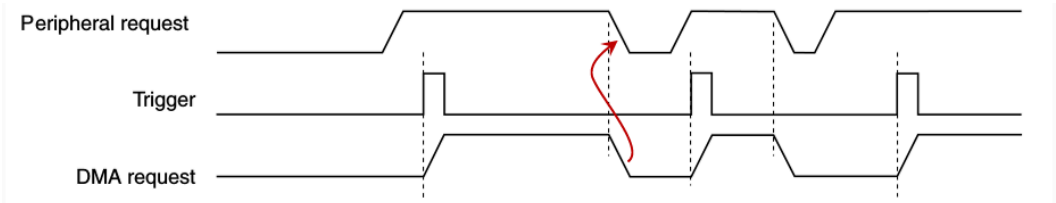


Figure 21-20. DMAMUX channel triggering: normal operation

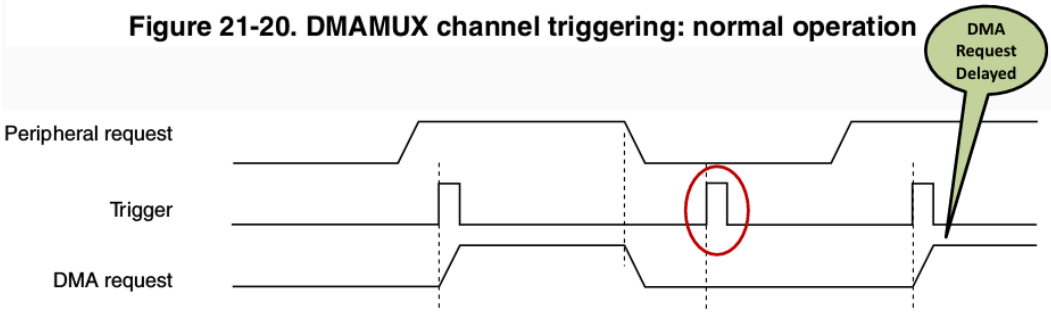


Figure 21-21. DMAMUX channel triggering: ignored trigger

⚙️ Registro DMAMUX_CHCFGn

- `ENBL`: Habilita el canal DMA.
- `TRIG`: Habilita el trigger periódico (modo Periodic Trigger con PIT).
- `SOURCE[5:0]`: Número de slot del periférico fuente (0..51). Ver tabla de slots en el Reference Manual del K64.

```
// Canal 0, fuente = SPI (slot 16), con trigger periódico del PIT
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(16)
                  | DMA MUX_CHCFG_TRIG_MASK;

// Canal 0, fuente = PORTC (slot 51), modo normal (GPIO)
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(51);

// Canal 0, fuente = Always-enabled (slot 63), con trigger del PIT
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_TRIG_MASK
                  | DMAMUX_CHCFG_SOURCE(63);
```

Tipos de Fuentes DMA

Tipo de Fuente	Cant. de Canales	Características
Gated (Periféricos)	Slots 0..41	Throttled. El request queda gateado hasta que el periférico lo activa. Permite trigger periódico con PIT (canales 0..3).
Always-Enabled	10 slots (ej. slot 63)	Sin throttling. Transferencias rápidas: memoria→memoria, GPIO rápido, activación por software.

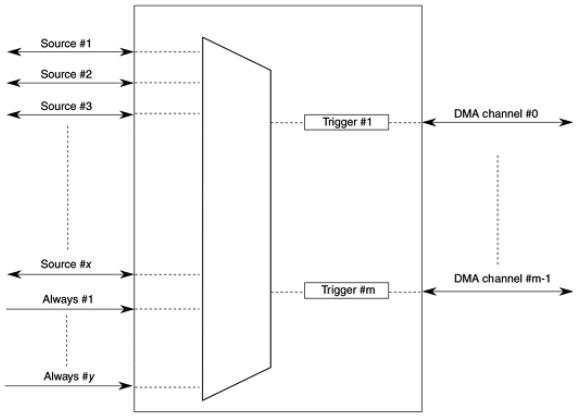


Figure 21-19. DMAMUX triggered channels

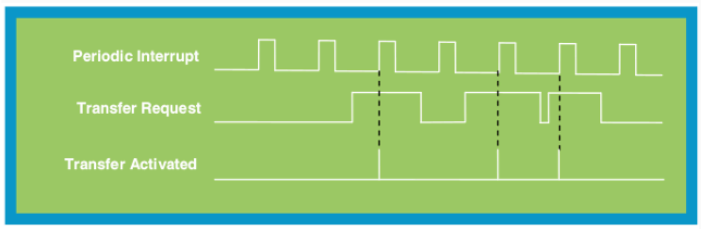


Figure 4. PIT gated transfer activation (Throttled)

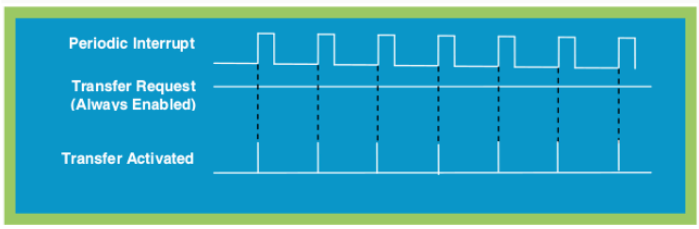


Figure 5. PIT only transfer activation

TCD: Transfer Control Descriptor

El TCD es la estructura que describe completamente cada transferencia DMA. Cada canal del eDMA tiene su propio TCD de 32 bytes en una memoria local:

```
typedef struct {
    uint32_t SADDR;          // Source Address (dirección de origen)
    uint16_t SOFF;           // Source Offset (offset por transacción menor)
    uint16_t ATTR;           // Transfer Attributes (tamaño, SMOD, DMOD)
    uint32_t NBYTES_MLNO;    // Bytes por minor loop (por cada DMA request)
    uint32_t SLAST;          // Ajuste de SADDR al finalizar el major loop
    uint32_t DADDR;          // Destination Address (dirección de destino)
    uint16_t DOFF;           // Destination Offset (offset por transacción menor)
    uint16_t CITER_ELINKNO;  // Contador actual del major loop
    uint32_t DLASTSGA;       // Ajuste de DADDR / puntero al siguiente TCD (S&G)
    uint16_t CSR;            // Control and Status (INTMAJOR, ESG, DREQ, etc.)
    uint16_t BITER_ELINKNO;  // Contador inicial del major loop (= CITER al inicio)
} TCD_t;
```

💡 Major Loop y Minor Loop

- **Minor Loop:** Una sola transacción de `NBYTES` bytes. Se ejecuta ante cada request del periférico.
- **Major Loop:** Conjunto de `CITER` minor loops. Al completarse el major loop, el DMA puede:
  - Generar una interrupción (`CSR[INTMAJOR] = 1`).
  - Ajustar `SADDR` sumando `SLAST` (para wrap-around de buffer circular).
  - Cargar un nuevo TCD en modo Scatter-Gather (`CSR[ESG] = 1`).

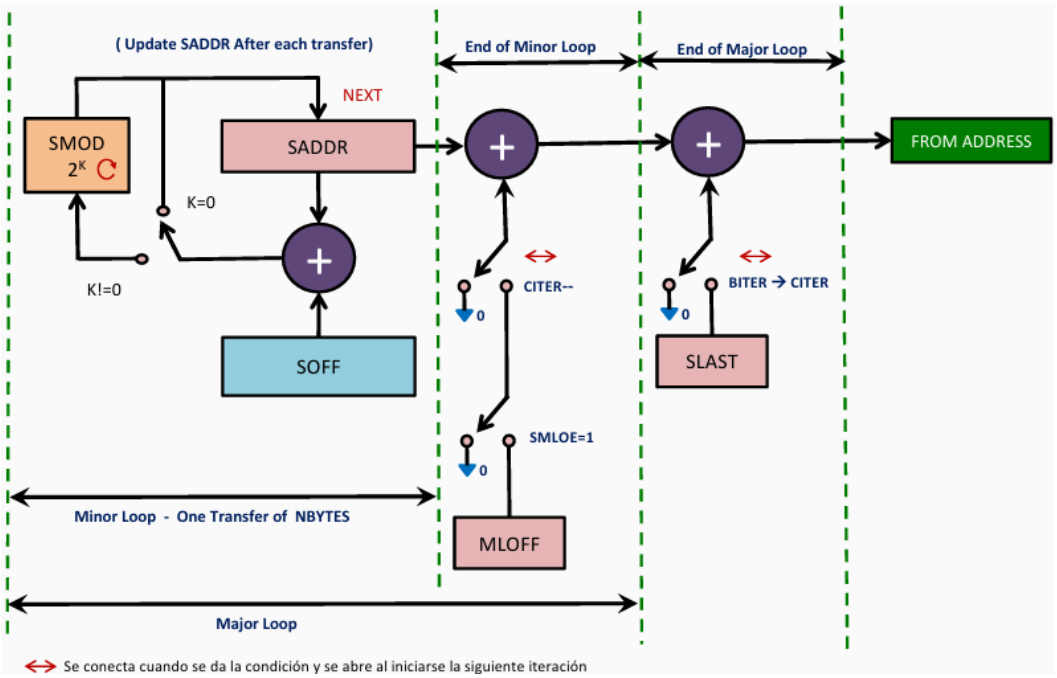


Figura 37: Source Address Generation Diagram

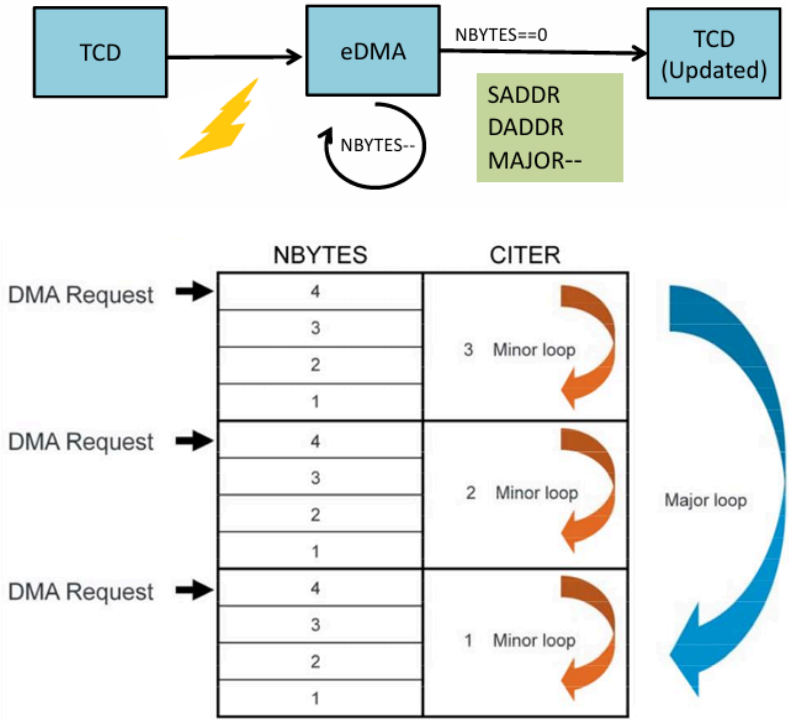


Figura 39: Transferencia de 4 bytes por minor loop

3. 3. 5. 1 Campos Principales del TCD

Campo	Descripción
`SADDR`	Dirección de inicio de la fuente de datos.
`SOFF`	Offset que se suma a `SADDR` tras cada minor loop. `0` = dirección fija (ej. registro de periférico).
`DADDR`	Dirección de inicio del destino de datos.
`DOFF`	Offset que se suma a `DADDR` tras cada minor loop. `0` = destino fijo.

Campo	Descripción
`ATTR`	`SSIZE`/`DSIZE`: tamaño de dato (`0`=8bit, `1`=16bit, `2`=32bit). `SMOD`/`DMOD`: habilita buffer circular (ver más abajo).
`NBYTES`	Número de bytes transferidos por minor loop (por cada request del periférico).
`CITER`/`BITER`	Contador actual e inicial del major loop. **Deben inicializarse con el mismo valor.**
`SLAST`	Valor sumado a `SADDR` al finalizar el major loop. Negativo para wrap-around: `SLAST = -sizeof(buffer)`.
`DLASTSGA`	Ajuste de `DADDR` al finalizar el major loop. En modo Scatter-Gather apunta al próximo TCD.
`CSR`	Control: `INTMAJOR` = interrupción al fin del major loop; `ESG` = scatter-gather; `DREQ` = deshabilitar request al finalizar.

3. 3. 5. 2 Buffer Circular con SMOD / DMOD

El campo `SMOD` (o `DMOD`) en `ATTR` define un **buffer circular por hardware**: los K bits menos significativos de `SADDR` quedan confinados en un buffer de tamaño $M = 2^K$. La dirección base debe ser múltiplo de 2^K.

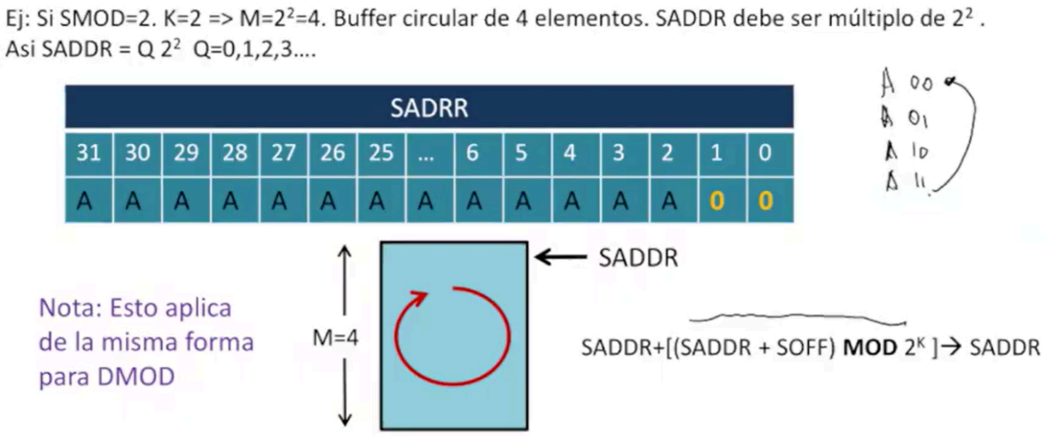


Figura 40: Ejemplo de buffer circular

```
// Buffer circular de 4 elementos de 1 byte: SMOD=2 → M = 2^2 = 4
DMA0->TCD[0].ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_SMOD(2);
```

Ejemplo Práctico: DMA + PIT → GPIO (Toggle de Pin)

Generar una señal cuadrada en `PTC0` escribiendo desde una tabla en memoria al registro `PTOR` del puerto C, con el PIT como trigger periódico a $5.1\ \text{ms}$:

🛠️ DMA + PIT → GPIO Toggle (PTC0)

```
// Tabla de valores a escribir en PTOR (Toggle Output Register)
static uint32_t sourceBuffer[] = {0x0001, 0x0001, 0x0001, 0x0001};

// 1. DMAMUX: canal 0, always-enabled (slot 63), trigger periódico del PIT
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_TRIG_MASK
                  | DMAMUX_CHCFG_SOURCE(63);

// 2. TCD0
DMA0->TCD[0].SADDR         = (uint32_t)(sourceBuffer);
```

```
DMA0->TCD[0].DADDR         = (uint32_t)(&(PTC->PTOR));
DMA0->TCD[0].SOFF          = 0x04;   // Avanzar 4 bytes por transacción
DMA0->TCD[0].DOFF          = 0x00;   // Destino fijo (siempre PTOR)
DMA0->TCD[0].ATTR          = DMA_ATTR_SSIZE(2) | DMA_ATTR_DSIZE(2);
DMA0->TCD[0].NBYTES_MLNO   = 4;
DMA0->TCD[0].CITER_ELINKNO = DMA_CITER_ELINKNO_CITER(4);
DMA0->TCD[0].BITER_ELINKNO = DMA_BITER_ELINKNO_BITER(4);
DMA0->TCD[0].SLAST         = -(int32_t)sizeof(sourceBuffer);

// 3. PIT canal 0: período = 5.1 ms
PIT->MCR = 0x00;
PIT->CHANNEL[0].LDVAL = 256000 - 1;  // 256000 × 20 ns = 5.12 ms
PIT->CHANNEL[0].TCTRL = PIT_TCTRL_TEN_MASK;

// 4. Habilitar request del canal DMA 0
DMA0->ERQ = DMA_ERQ_ERQ0_MASK;
```

Ejemplo Práctico: DMA + FTM → PWM Variable desde Tabla

Generar una señal PWM cuyo duty cycle varía automáticamente usando una tabla en memoria. El FTM solicita la transferencia DMA en cada coincidencia de canal (`CHnF`):

🛠️ DMA + FTM0 → PWM con Duty Cycle Variable

```
uint16_t sourceBuffer[] = {1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000};

// DMAMUX: canal 0, fuente FTM0_CH0 (slot 20), modo normal
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(20);

// TCD0: tabla de duty cycles → registro CnV del FTM0 CH0
DMA0->TCD[0].SADDR         = (uint32_t)(sourceBuffer);
DMA0->TCD[0].DADDR         = (uint32_t)(&(FTM0->CONTROLS[0].CnV));
DMA0->TCD[0].SOFF          = 0x02;   // Avanzar 2 bytes (uint16_t)
DMA0->TCD[0].DOFF          = 0x00;   // Destino fijo (siempre CnV)
DMA0->TCD[0].ATTR          = DMA_ATTR_SSIZE(1) | DMA_ATTR_DSIZE(1);
DMA0->TCD[0].NBYTES_MLNO   = 0x02;
DMA0->TCD[0].CITER_ELINKNO = DMA_CITER_ELINKNO_CITER(8);
DMA0->TCD[0].BITER_ELINKNO = DMA_BITER_ELINKNO_BITER(8);
DMA0->TCD[0].SLAST         = -(int32_t)sizeof(sourceBuffer);
DMA0->TCD[0].DLASTSGA      = 0x00;

// FTM0: PWM con DMA habilitado en CH0
FTM_SetPrescaler(FTM0, FTM_PSC_x32);          // Prescaler ×32
FTM_SetWorkingMode(FTM0, 0, FTM_mPulseWidthModulation);
FTM_SetPulseWidthModulationLogic(FTM0, 0, FTM_lAssertedHigh);
FTM_SetModulus(FTM0, 10000 - 1);
FTM_SetCounter(FTM0, 0, 5000);
FTM_DmaMode(FTM0, FTM_CH_0, FTM_DMA_ON);      // DMA ON en CH0
FTM_StartClock(FTM0);
```

Con prescaler $32\times$ y Bus Clock $50\ \text{MHz}$: $T_{\text{PWM}} = \left(\frac{1}{50\ \text{MHz}}\right) \times 32 \times 10000 = 6.4\ \text{ms}$

Modos Avanzados del DMA

3. 3. 8. 1 Scatter-Gather

El modo **Scatter-Gather** permite encadenar múltiples TCDs en una **lista enlazada circular**: al finalizar el major loop de un TCD, el hardware carga automáticamente el siguiente TCD desde memoria principal sin intervención de la CPU.

Esto se usa para implementar buffers de tamaño arbitrario (no potencia de 2) o esquemas de doble buffer (**ping--pong**):

```
// Encadenamiento circular: al terminar TCD_A se carga TCD_B y viceversa
TCD_A.DLASTSGA = (uint32_t)&TCD_B;
TCD_B.DLASTSGA = (uint32_t)&TCD_A;
```

3. 3. 8. 2 Channel-to-Channel Linking

El **Channel Linking** permite que al finalizar un minor loop (o major loop) de un canal DMA, se active automáticamente otro canal DMA. Se usa para encadenar operaciones secuenciales complejas completamente por hardware.